



**Utrecht
University**

Editing the editor: proposing an extensible language extension for Agda

Author:
Job Vonk

First Supervisor:
Dr. W. S. Swierstra

Second Supervisor:
I. G. de Wolff MSc.

Daily Supervisor:
L. B. Chonavel MSc.

July, 2026

Table of Contents

1. Introduction	1
2. Background	3
2.1. Visual Studio Code	3
2.2. Language Server Protocol	4
2.3. Existing Agda extensions	5
2.3.1. Implementations	6
2.4. Metaprogramming	7
2.4.1. In Idris1	8
2.4.2. In Agda	8
2.4.3. In Lean4	9
3. Building Avea	10
3.1. Anatomy of a Visual Studio Code extension	10
3.2. Sending interactions	12
3.3. Receiving and handling responses	13
3.4. Reacting to changes from the user	17
3.5. Input mode	17
4. Evaluation	19
4.1. Importing Javascript libraries	19
4.2. Handling asynchronicity	20
4.3. Mitigating stack overflows	21
5. Adding extensibility	23
5.1. Start of the Avea extension EDSL	26
5.2. Edit-time actions in Agda	27
5.3. Changing Agda's meta-language	32
6. Conclusion	34
7. Future work	35
7.1. Verifying parts of the extension	35
7.2. Making the extension editor-agnostic	35
7.3. Improving Agda's Foreign Function Interface	35
7.4. Port of Lean4's user widgets to Avea and Agda	37
8. Bibliography	39

1. Introduction

Dependently-typed programming languages provide a means to prove the absence of bugs. This gives rise to code bases where the functionality is completely verified, that is, the use of the library or a program cannot go wrong according to their defined specification. Even though complete verification sounds ideal for all code bases, there is one major flaw: using formal methods, like dependently-typed languages and theorem provers, to verify code can be intensive. It takes mathematically skilled people, and quite some time (compared to informal quality assurance methods like testing) to fully verify a system [1].

This is because dependent types can be much more precise about the programs they describe. The type can encode invariants that must hold during the function’s execution, or preconditions on the parameters. Compared to non-dependently typed languages, writing programs in dependently typed languages without tools for querying the types of terms or holes is difficult. Therefore, many dependently typed languages need interactive development environments (IDEs) to be useful to programmers.

Additionally, dependently typed languages are also often used as theorem provers. However without tooling, such a system can be hard to prove theorems in, especially since normalisation of the system can be hard to reason about without querying the system. Additionally, these systems benefit even more from an IDE – which for theorem provers is often called an *interactive theorem prover* (ITP) – since an ITP can also provide helpful tooling to construct new proofs.

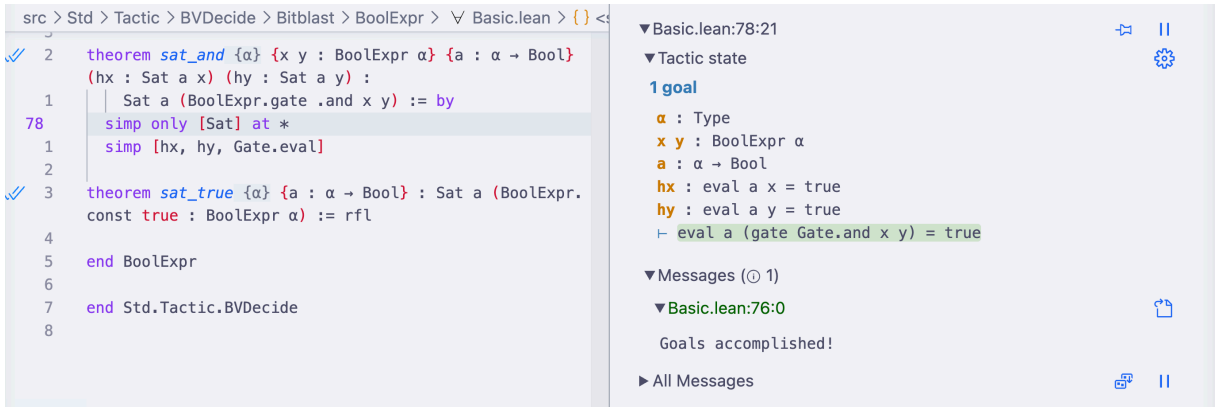


Figure 1: Example of using Lean4, an interactive theorem prover, to prove a theorem. The panel on the right shows information about the current state of the proof and what remains to be done based on the location of the cursor.

Juhošová et al. [2] have investigated what obstacles students experience when learning Agda as an interactive theorem prover. Their advice is to improve robustness and accessibility, where one point they mention for both improvement areas is polishing and improving the existing tooling.

Juhošová et al. [3] also conducted a study about type driven development. The participants of the study were asked to give improvements to languages that offer type-driven development and their tooling. Some of the improvements the participants suggested were “wider editor support for interactive modes”, “better IDE support for large-scale proof engineering” and more refactorings. Backx [4] has also investigated how novice dependently-typed language programmers write code, but also includes experienced programmers in his study. He, too, saw that users experience a lack of tooling support for refactorings, and also recommends authors of dependently-typed interactive theorem provers to improve the editor tooling with, for example, a search feature for lemmas, or refactoring features that can also be found in non-dependently typed languages.

Focusing on Agda in particular, even though there is strong incentive from users to develop new features, we see little innovation in editor features: all of the extensions for all currently available editors support roughly the same feature set. These editor features are provided by the compiler, and the extensions all merely implement an interface to these compiler features from an editor. One reason for the lack of innovation might be that extending this feature set is quite difficult to do: the features are implemented within the Agda compiler, which is a large Haskell code base. Additionally, these features are quite tightly coupled to Agda's elaborator and unification system, which are complex systems that are hard to interact with without much prior experience.

Another reason for the little innovation might be the fact that none of the extensions are written in Agda itself, rather they are implemented in various languages like ReScript, Haskell or Emacs Lisp to name a few. This raises the barrier of entry for people familiar with Agda wanting to contribute a new extension feature.

These two observations motivate the following research questions:

1. Is it possible to make an extensible Agda extension by writing it in Agda itself?

We will show this by writing an Agda language extension for Visual Studio Code in Agda that is extensible by adding extra new features onto it, which can also be written in Agda. Evaluating the extensibility of the extension is quite difficult, since there exists little comparison material. One rudimentary way to evaluate the extensibility of the extension is by measuring the number of extra lines of code required to add new features.

2. Is Agda a suitable language to write practical programs with?

Current big agda projects are libraries like the standard library, `unimath`¹ and `cubical`², which are impressively large and highlight Agda's strength as a theorem prover. However, at the Agda wiki's home page, the language is introduced as a dependently-typed *programming language* before being introduced as a proof assistant³. Strangely, there does not seem to be a big practical programming project showing Agda's viability as a programming language. Writing a language extension in Agda uncovers whether or not Agda is a viable *programming language* as opposed to a theorem prover. After the development, we can reflect on issues in the language or its tooling, and report a list of improvements to make Agda more suitable for practical applications.

¹<https://unimath.github.io/agda-unimath/>

²<https://github.com/agda/cubical>

³<https://agda.readthedocs.io/en/latest/getting-started/what-is-agda.html>

2. Background

2.1. Visual Studio Code

Visual Studio Code is an open-source code editor developed by Microsoft. According to last year's StackOverflow Developer survey, 79.6% of the respondents used Visual Studio Code regularly in 2025 [5]. While the editor ships with support for Javascript/Typescript support out of the box, it has a robust extension system and a marketplace with extensions for many other programming languages. This includes extensions for mainstream languages like C++, C#, Rust, etc. However, Visual Studio Code has also been target for dependently-typed languages. In particular Lean4 has builtin support for LSP, and has a very sophisticated extension.

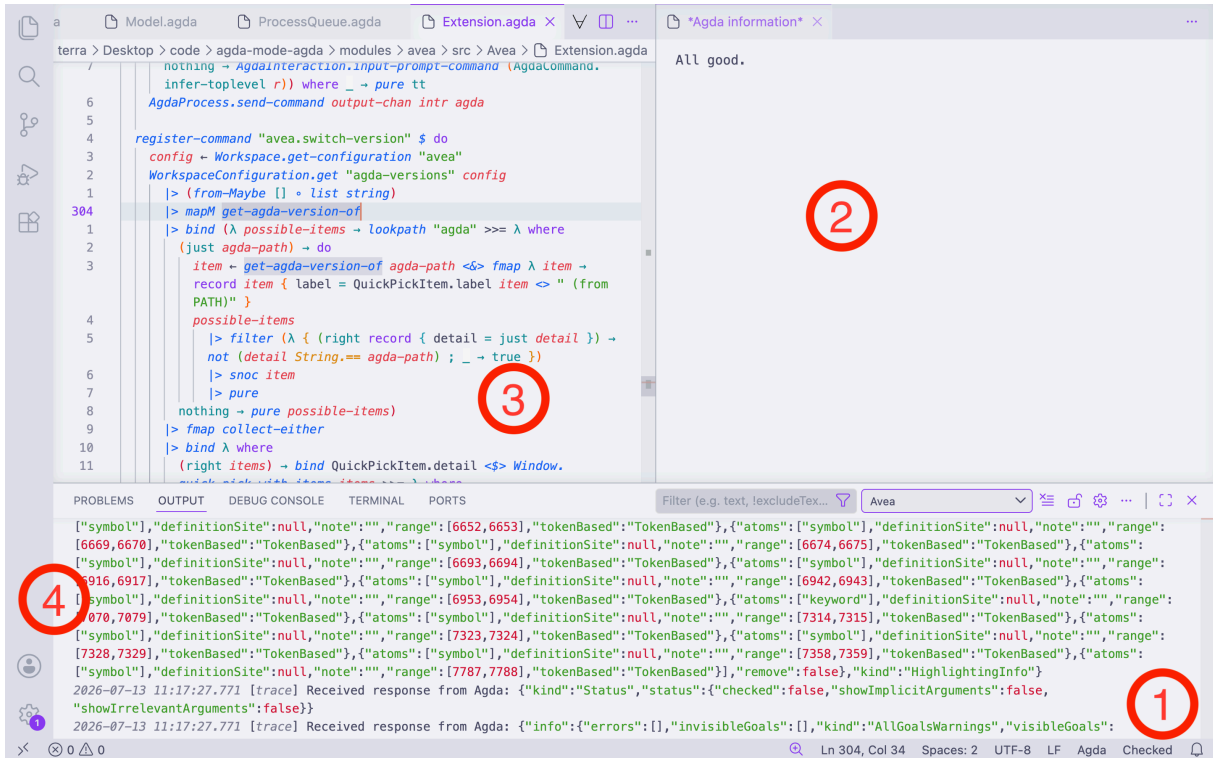


Figure 2: Screenshot of Visual Studio Code, the numbers correspond to the enumeration in the paragraph below.

Extensions can be written in Javascript or Typescript. When VS Code executes the code for an extension, it provides an API which allows extensions to interface with the different parts of the editor. Some examples of interacting elements from an extension are highlighted in Figure 2.

1. There is a status bar at the bottom of the screen that displays small amounts of information – typically single words or icons. Commonly, the status bar contains the the location of the cursor, how many errors the project contains or the language of the currently active buffer.
2. The API allows extension developers to create panels, these are implemented as an HTML iframe. This means that the panel can display arbitrary HTML, CSS and Javascript. Additionally, the extension can send and receive messages to and from the panel using the window's `postMessage` method and the window's message event.
3. The editor itself can also be interacted with. Extension developers can request parts of the buffer based on positions within the document. Or the buffer can be programmatically edited, or decorated using syntax highlighting.

- Extensions can also log information to VS Code’s output window. The API for this looks like a simple logging library, with different levels of logs. The logs can be viewed in the output window per extension, and can also be exported from there. This is especially useful during debugging.

2.2. Language Server Protocol

The Language Server Protocol (LSP) is a protocol developed by Microsoft that aims to standardise the communication between language tooling and integrated development environments (IDEs) [6]. The purpose of the protocol is to solve the $M \times N$ problem for language extensions. The essence of this problem is that language tooling developers need to implement a separate extension for each editor they want to support. As shown in Figure 3, given N languages and M editors, this results in $M \times N$ extensions in total. LSP simplifies this by letting language tooling developers only implement the language side of LSP, and editor developers implement the editor side of LSP. This reduces the total number of extension down to just $M + N$, which is significantly less burden on tooling developers.

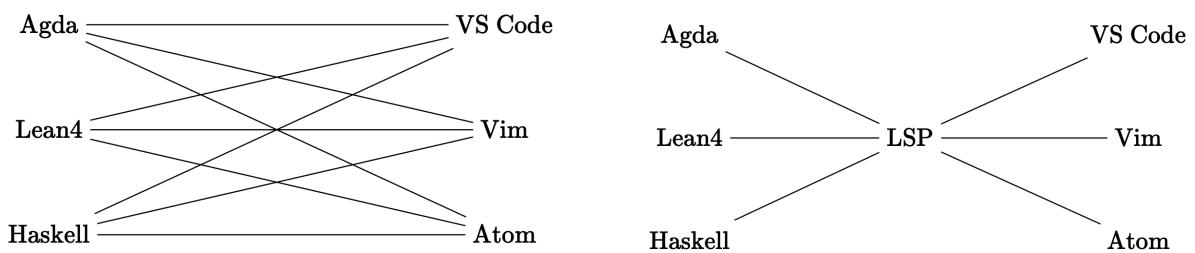


Figure 3: Visualisation of how LSP solves the $M \times N$ problem for language editors: each language tooling author needs to support just LSP instead of every single editor.

In more technical detail, the editor sends notifications to the language tool about actions the user has done, e.g. opening or closing a document, changing the source code, or executing a code action. The language tool can respond to those notifications as well as send notifications to the IDE about warnings, errors or definition sites.

The LSP protocol has been proven in practice by many main stream languages, among which C++, C# and Rust. Even dependently-typed languages have implementations for the LSP protocol:

- Idris has a Visual Studio Code extension that supports the LSP protocol [7]. There is a package `idris2-lsp` that depends on the Idris compiler. Since both the LSP server and the compiler are implemented in Idris itself, the LSP can depend on the compiler directly. And therefore, the LSP can immediately invoke the type checking commands. The Idris LSP has some limitations, however. One of those is that hover actions only work if the entire file type checks.
- Lean4 has an implementation of the LSP protocol with Lean-specific extensions. The implementation is written in Lean itself as part of the compiler [8]. The Visual Studio Code extension, for example, can rely on the core LSP part for providing autocompletions and code actions. However, since the extensions are non-standard, they require implementations for every IDE separately. For example, the `vscode-lean4` extension has workspaces for setting up the LSP, as well as the `infoview` window, which displays context information, results of `#eval`s and goal types.
- Rocq has two extensions for Visual Studio Code, each with their own implementation of the LSP protocol: `vsrocq` with the `vsrocq-language-server` [9], and `rocq-lsp` [10]. The `rocq-lsp` GitHub repository describes that, in addition to the standard protocol, there are interactions specific to Rocq for, for example, displaying the proofs goals in tactic mode and file checking progress updates. Similar to Lean, both extensions also feature a `infoview` window where the proof goals, context and `Print` statements are displayed.

The extensions these languages have made are all quite similar of nature, and Rask et al. [11] have therefore developed the Specification Language Server Protocol (SLSP) that aims to standardise extensions to the LSP protocol regarding theorem proving, providing goal info and translating to other languages. They have implemented a server for SLSP for the Vienna Development Method, which is a specification language that also features theorem proving.

2.3. Existing Agda extensions

The Agda ecosystem has a number of extensions for different programming editors. All of these extensions behave very similarly and feature a similar set of tools to Agda developers:

- Agda has many different **commands** that users can invoke using keybinds with their cursor at certain locations. These commands can be categorised in roughly three categories.

Goal refinement commands allow the user to enter (parts of) expressions into a goal and Agda will type-check or refine them based on the goal's type. Examples of commands in this category are:

- the refine command, which inserts question marks for every missing argument of the given expression;
- the make case command, which creates exhaustive pattern matching clauses for a parameter, where possible. For example, when pattern matching on a constructor, make case will create a clause matching constructor.

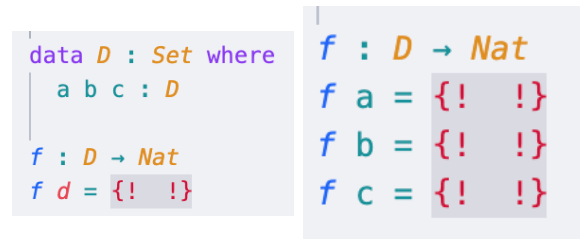


Figure 4: Example use of the make case command: in a goal the user can type C-c C-c in a goal (left image), and specify which parameters to case split on and all clauses appear (right image).

Goal display commands give information about the goal under the cursor, which is displayed in the display buffer. This information can help the user see what they need to prove and what proofs are available in the current context. Examples of commands in this category are:

- the infer command, which displays the inferred type of the goal within an expression;
- and the context command, which displays the available local variables and their types, as illustrated in Figure 5.

General Commands are commands that can be executed on a module or an arbitrary expression. Examples include

- the load command, which type checks the current module and returns syntax highlighting;
- the search about command, which searches the current module scope for functions with signatures containing some key types;
- the compile command, which compiles the current module using one of the available backends.
- The **display buffer** is a panel accompanying the editing buffers, where information about the state of the extension is displayed. For example, when the load file command is invoked the compiler shares information about its progress type-checking the current file and its dependencies. After type-checking completed, the buffer can show either information about goals – if the type-checking was successful – or it can show information about type errors in the current file or its dependencies.

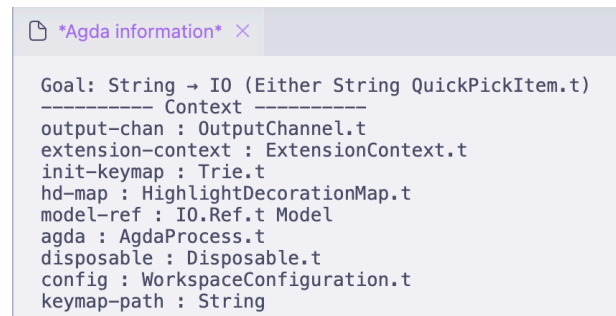


Figure 5: The output on the display buffer after invoking the context command (C-c C-.) in Avea.

- **Syntax highlighting:** Once the current file is successfully type-checked, the compiler also shares highlighting information to the extension. Agda provides *semantic highlighting tokens*, which means that the tokens are coloured based on what they represent. For example, instead of colouring a variable black regardless of its value – Agda will colour a variable referencing a function blue. Additionally, Agda provides syntax highlighting for different error types, and the locations of where they occur.

```
data t : Set where u : t

f : t → T
f u = tt
```

Figure 6: Semantic syntax highlighting in Avea. For example, the colour of the constructor `u` is different from the type `t`, even though both are variables.

- **Jump to error:** if the type-checking was not successful, then the compiler will point to the type-error it got stuck on. The extension receives information about where this error is located and moves the user’s cursor to that file at the correct position.
- **Go to definition:** The extension answers go-to-definition requests from the editor, where definition site information is provided by the `HighlightingInfo` message. Visual Studio Code provides a position, and the extension has to query the tokens for the currently active file to find one which has the position in its range.
- **Input mode:** As mentioned in Section 2.3, all extensions have support for the unicode input mode to type unicode characters in Agda source code. This extension also has support for the input mode, in a similar fashion as the emacs-mode, where the status bar shows which characters can be typed next, as well as show which characters can currently be selected using the arrow keys.

```
⇒
|r[-=abcdefghijklrt~] > →, ⇒, ⇨, [⇒], ⇨, ⇨, ⇨, ⇨, ⇨, ⇨, ⇨, ⇨
```

Figure 7: The status bar during input mode shows which characters have been typed, which characters can be typed next, and which variants of the current character there are.

2.3.1. Implementations

There are various implementations of Agda extensions for different editors. These implementations can use the endpoints provided by the Agda compiler for external programs to call Agda internals. The compiler has flags which open a Read-Eval-Print-Loop (REPL) environment that extensions can use to issue commands and receive responses. In general every extension has a similar high-level architecture, which is illustrated in Figure 8.

Every extension spawns one or more processes of the REPL the Agda compiler provides. When the user interfaces with the editor, the extension is notified and sends requests to Agda. Agda responds and the extension makes sure the correct updates are done to the editor.

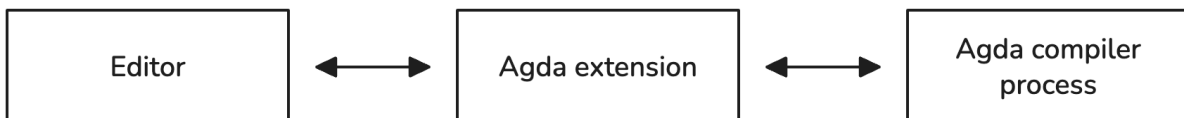


Figure 8: High level architecture of all Agda extensions. The extension is the mediator between the editor and the Agda compiler process.

Below we discuss the existing Agda extension implementations for various editors, and highlight some differences in their implementation.

- emacs-mode is the first Agda extension, which was developed alongside the Agda compiler. The extension is divided into two parts:

1. The user event listeners, these are functions triggered by command hooks within Emacs. These hooks send different commands to the Agda compiler, depending on the type of action. The extension opens a handle to an Agda process with the `--interaction` flag, that the extension communicates with.
2. Agda responds with Emacs lisp code that triggers response functions. These response functions are parameterised by data that is provided by the Agda compiler, which encodes the response data in Emacs lisp data types.

```
(agda2-info-action
  "*All Goals*"
  "?0 : String → IO (Either String QuickPickItem.t)"
  nil)
((last . 1) . (agda2-goals-action '(0)))
```

Figure 9: Agda’s `--interaction` flag responds with Emacs Lisp code, which is executed by the emacs-mode extension. The `agda2-info-action` function updates the display buffer with the parameters filled in by the Agda compiler.

- Cornelis is an extension for Vim written in Haskell. As opposed as emacs-mode, it uses the `--interaction-json` flag, which has the same API as the `--interaction` flag, but responds with a JSON format instead of Emacs lisp code.

Cornelis also has a different way of handling responses and managing the communication to the Agda compiler. It creates a process handle for each buffer, such that responses can be processed asynchronously, and disregard the editor’s state.

- Agda-mode-vscode by banacorn is the original extension for Agda in Visual Studio Code (VS Code). It is written in ReScript, an OCaml-like language that compiles to JavaScript. It uses the `--interaction` compiler flag, since it predates the `--interaction-json` flag.

This extension tries to emulate the behaviour of the emacs-mode, but VS Code has less builtin machinery for tokenisation especially when the buffer has been edited. This means the extension listens for edits to the buffer, and recalculates the positions of the known tokens to keep the highlighting working for unchecked states. Additionally, the extension also features a system to facilitate the installation of Agda.

The VS Code extension also has integration with the Agda Language Server (ALS). It is a language server for Agda created by the same author as the VS Code extension. The implementation currently only contains a hover functionality that shows the type of a term once the file has successfully been type-checked.

- Recently, a new VS Code extension for Agda has been released by Benjamin Driscoll⁴. This extension is written in Typescript – the native language of the VS Code APIs. The functionality of this extension is mostly the same as the banacorn’s VS Code extension, apart from some bug fixes and user interface changes.

2.4. Metaprogramming

Metaprogramming refers to the process of writing programs that modify or generate other programs. In this thesis we mainly focus on static metaprogramming, that is metaprograms which evaluate at compile-time to syntax trees, which are used for compilation or further elaboration. Listing 1 shows a contrived example metaprogram in Agda. The metaprogram on the left builds up the syntax tree for an identity function, which can be spliced in using `unquote` as seen on the right.

⁴<https://github.com/willtunnels/agda2-vscode>

```

1 mk-Id : TC Term
2 mk-Id = returnTC
3   (lam hidden
4     (abs "A"
5       (lam visible
6         (abs "x"
7           (var 0 []))))))

```

```

1 id : {A : Set} → A → A
2 id = unquote mk-Id
3 -- Expands to the following
4 -- at type-checking time
5 id = λ {A} x → x

```

Listing 1: Example of a metaprogram in Agda. The function on the left defines how to build up the term for the identity function, the function on the right uses the metaprogram to define the identity function.

2.4.1. In Idris1

Idris1 has static metaprogramming [12]. Terms can be quoted, which converts the term to a Idris1 data type representing Idris1 terms, called `TT` (Listing 2). This data type also has a constructor for binders, which can create let bindings to introduce new variables. There is also an inverse of quotation, called unquotation. The term `%runElab m`, where `m : Elab TT` evaluates the metaprogram and the returned term of type `TT` is converted back into a unquoted Idris1 expression, which is later elaborated or compiled.

Metaprograms in Idris1 are written using a tactic-based embedded domain specific language (EDSL). The tactics are functions that update the “proof” state, which comprises among other technicalities, the hole queue and the incrementally built proof term. The tactics are defined from within the compiler and have the ability to invoke internal function in the elaborator. For example, the `fill` tactic used in Listing 3 places another quoted expression in the first hole of the hole queue and unifies its type with the expected type of the hole. If this fails, the compiler presents the user with a type error.

2.4.2. In Agda

Similar to Idris1, Agda also has static metaprogramming. Agda has data types for terms, clauses and definitions, among others. The `Term` data type reflects the structure of elaborated terms in Agda. This means, for example, that `let` and `where` bindings are already substituted and cannot be inspected or generated using metaprogramming. This is generally not a problem, though. The output of metaprograms in Agda is not inspected (unless debugging metaprograms), but immediately spliced into the syntax tree and used for further elaboration.

Another difference with Idris1 is that Agda does not use a tactic-based EDSL. Metaprograms are written in the `TC` monad, whose definition is opaque. The reflection API that Agda exposes defines a number of primitive functions that all operate in the `TC` monad. This monad holds the environment, and the metavariable state, among other technicalities.

```

1 data Binder
2   : Type -> Type where
3   Let (ty, val : a) -> Binder a
4   ...
5 data TT = Bind TTName (Binder TT) TT
6         | TConst Const
7         | ...

```

Listing 2: A snippet of the definition of the `Binder` and `TT` data types in Idris1. These are used to represent Idris1 terms and are the result of quoting regular Idris1 expressions.

```

1 do
2   [x, y] <- apply `(plus) [False,
3     False]
4   solve
5   focus x; fill `(Z); solve
6   focus y; fill `(S Z); solve

```

Listing 3: An example taken from [12]. It demonstrates how to build up the expression $Z + S\ Z$ using the metaprogramming monad.

2.4.3. In Lean4

Lean4 is another dependently-typed language and interactive theorem prover [8]. The language allows metaprogramming in different ways in a Lean program:

- In Lean4, all top-level statements, including `def`, `inductive` are called commands. Besides these builtin commands, the user has the ability to define their own commands. The definition of a command includes a metaprogram of type `Syntax → CommandElabM Unit`. The function receives the abstract syntax tree of the argument to the command, and must produce some side effects, which include updating or querying the environment or performing IO operations. Listing 4 illustrates these components of a command.

```
1 #check 2 + 2
2 -- The syntax for "2 + 2" is the argument to the metaprogram.
3 -- The side effects are:
4 -- * the type of the term represented by the syntax is inferred, which is Nat;
5 -- * and "2 + 2 : Nat" is written to the infoview.
```

Listing 4: An example command in Lean4, and how Lean4 calls the metaprogram associated with the command.

- Macros can access the environment and the local metavariables.
- Tactics can access the environment, local metavariables and the proof state. The functionality of a tactic is defined using the `TacticM` monad, which expands directly upon the `TermElabM` monad.

A metaprogram in Lean4 looks similar to one in Agda, where you can call functions in a monadic non-tactic style. Whereas Agda only allows functions from the reflection API to be called, the Lean4 compiler is written in Lean itself, and allows metaprograms to directly call any function defined in the compiler. This means that metaprograms in Lean4 can use any part of Lean4’s elaborator.

Lean4 also has extensible user interface inspired by Korkut et al. [13], which allows user to execute metaprograms at edit-time and display their results in visualisation in the infoview [14]. Figure 10 shows an alternative implementation for the `#check` command, where the user can type an expression in the infoview (on the right) and the type of the expression is inferred and displayed. When the user puts their cursor on a `#widget` command in the source file, the `lean4-vscode` extension loads the widget’s React code into the infoview, from where the widget can call metaprograms (of type `TermElabM`) using the LSP to the compiler.

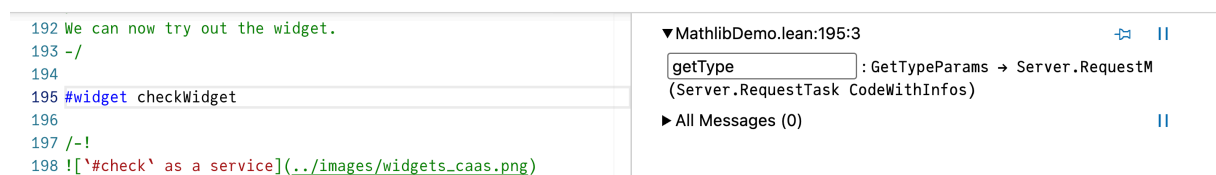


Figure 10: One of the example widgets defined on the Lean4 widgets documentation page⁵. The widget allows the user to enter an expression in the infoview and its inferred type is displayed.

⁵<https://lean-lang.org/examples/1900-1-1-widgets/>

3. Building Avea

One result of this thesis is a working implementation of a Visual Studio Code extension written in Agda – which compiles to Javascript – that supports the same set of features other Agda extensions also have.

The extension acts as the mediator between Agda and Visual Studio Code. Agda exposes an language-agnostic API with which the extension can communicate. The extension can communicate with Visual Studio Code through the use of the Javascript API. Figure 11 shows the high-level architecture of the extension and how the different parties communicate with each other.

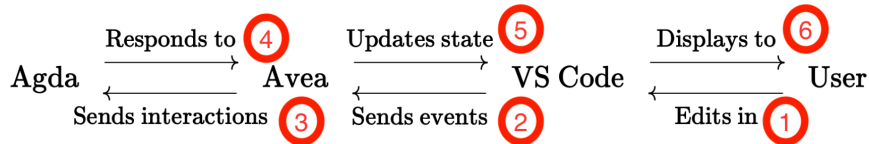


Figure 11: Communication patterns within the high-level architecture of the extension.

In order, the extension operates as follows:

1. The user makes an edit in the buffer in Visual Studio Code or triggers a keybind.
2. Visual Studio Code listens for keybinds configured by the extension, and invokes the handler that is registered in the extension.
3. In the handler, the extension defines what to send to the Agda process – which typically includes which file the interactions works on, and how Agda should respond – and awaits the response.
4. Agda responds with many different responses to the sent interaction, and notifies the extension that new interactions may be sent at the end. The responses are encoded in JSON, and have a kind, which can be used to distinguish what the response’s content is about. Responses can contain new highlighting tokens, information about the progress of type-checking, or formatted errors and warnings.
5. Each response is decoded into Javascript objects and then parsed according to the kind property. Each of the parsed responses is then handled individually. Handling a response typically involves updating the internal caches with potentially new locations for tokens and interactions points.
6. After the caches are updated, the extension sends change requests for the buffer or updating the display buffer via the Visual Code Studio API.

The sections below each elaborate in more detail about each of the steps described above. In particular, Section 3.1 discusses the activation of the extension and the registration of the handlers. Section 3.2 discusses how interactions are encoded and sent to the Agda process. Section 3.3 discusses how responses are parsed and handled. Section 3.4 discusses how edits in the buffer are handled, and why it is necessary to handle changes.

3.1. Anatomy of a Visual Studio Code extension

A Visual Studio Code extension should be written in Javascript. The package.json file lists the dependencies and the entry point file. The entry point file should contain two exported functions: an `activate` function and a `deactivate` function. The `activate` function is called once when the extension’s configured activation events is triggered. For Avea specifically, it is configured to activate the extension once an Agda file is opened by the user.

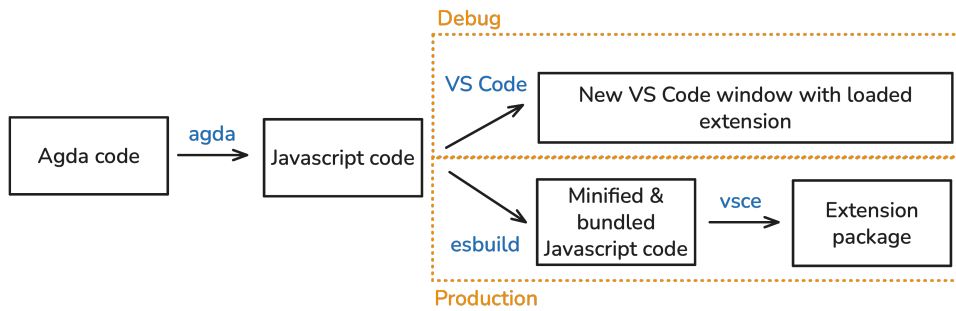


Figure 12: A high-level overview of the infrastructure required to run Avea, both in debug and production mode.

Visual Studio Code also provides ways to run your extension both in debug mode as in production mode. The debug mode can be triggered from VS Code’s debug menu and runs a special `extensionHost` debug profile, which opens a new window with the extension loaded into it. Any output on the standard output is redirected to the main window’s debug output, and additionally breakpoints can be set. To publish an extension, VS Code has a tool called `vsce`, which can package all the files needed to run the extension into a compressed binary VSIX file. This file is then published to the marketplaces.

One issue is that `vsce` only packages the files you tell it to package. Additionally, even though the `vsce` utility already compresses the extension files, but Visual Studio Code recommends to keep the source code for the extension itself as small as possible still. Following these guidelines, we cannot use put the `node_modules` folder – the folder containing all dependencies, both runtime and buildtime – in the extension package. For Avea, this folder can easily grow as large as a hundred megabytes, most of which is dedicated to build time tools. This would be a waste to include in the extension package, and therefore packaging requires some preprocessing step on the source code. For this, Avea uses another tool called `esbuild` – a bundler for Javascript – to combine the generated Javascript source code by Agda with the source code of just the runtime dependencies in `node_modules`, and to minify the resulting bundle, which is then put in the extension package. Figure 12 shows a diagram of how the tools call each other.

Once the extension is loaded – either in a debug window or using the production extension package – the code in the entry file is executed. The entry file sets up the dependencies – discussed in more detail in Section 4.1 – and then calls the activation function in Avea. This activation function has two purposes: it creates all of the resources that need to be created in an IO context, and it registers all of the handlers for keybinds, commands, hover-, semantic token and definition providers. The `activate` function serves a very similar purpose as the `main` function does in regular Agda programs, where all IO operations have to happen in `activate` or a function called from there.

The handlers for the keybinds and commands and such keep listening until they are explicitly deactivated. It is the extension’s responsibility to shut down the event listener or kill the process every time a long-running event handler or process is started. When creating handlers for events in the VS Code API, they return a `Disposable`, which is simply an object that has a `dispose` method. `Disposables` can be registered in the extension’s context object, which is the only parameter in the entry file’s `activate` function. After registering, VS Code will run all the `dispose` functions registered in the context, on *deactivation*.

TODO: Provide the Agda encoding of `Disposables` in Avea.

Deactivation happens just before the extension should no longer be active, for example, when a non-Agda file is opened, or the extension is uninstalled or disabled, depending on the activation events.

Besides an `activate` function in the entry file, there is a `deactivate` function, which is called on deactivation, and can be used to define cleanup that cannot be defined using `Disposables`.

3.2. Sending interactions

```
1 register-command "avea.make-case" $ do
2   model ← IO.Ref.get model-ref
3   just intr ← AgdaInteraction.under-cursor-command model AgdaCommand.make-case
4   where _ → pure tt
5   AgdaProcess.send-command output-chan intr agda
```

Listing 5: Implementation of the handler for the `make-case` command in Avea’s `activate` function. It sends an interaction to the Agda process containing the current file and the location of the goal under the cursor.

Using the implementation of the `make-case` command in Avea shown in Listing 5 as an example, a command registration consists of 3 parts:

1. The `register-command` (line 1) function calls the VS Code API to register a new handler under a given name.
2. In the `package.json` file, it should be declared what keybinds or commands should call the handler by name.
3. In the handler, we define what the keybind or command should actually do. Depending on the kind of command, the handler does the following:
 - If the command only updates the internal cache (called `model-ref` in Listing 5), or needs to communicate to VS Code, then the handler can use the API directly.
 - If the command needs to interact with Agda, then an `AgdaInteraction` should be created and this can be sent to the Agda process (line 3-5). The handling of an Agda interaction is done in Response modules (described in more detail in Section 3.3).

To send an interaction to Agda, a child process must be set up first. Avea makes sure to have one child process running at all times, so that Agda can keep its internal state about, for example, previously loaded modules. To start and manage the process, the following steps are involved:

1. Before the process is spawned, Avea needs to read the settings via the VS Code API, to read the path to the Agda binary. The user can configure multiple paths to different Agda binaries, which can be selected using a keybind within Avea. For each of the binaries, the version is requested, which is used for display and to check whether the binary exists. When the user selects one of the binaries, the Agda path setting is set to that path, which is used when spawning the child process.
2. Sending interactions to Agda is done by spawning a child process running Agda’s `--interaction-json` mode, and sending data over the child process’s standard input. Avea manages a single Agda process, which is spawned on activation.
3. Once the process is spawned, Avea attaches different handlers. A data handler receives the output from the Agda process, which does the parsing and processing (detailed in Section 3.3). An error handler listens for unexpected closes of the process. This handler reports the error to the user, with the ability to restart the process.

Lastly, since the child process library is not part of the VS Code API, Avea manually creates a `Disposable` that kills the process gracefully – removing the attached event listeners as well. **TODO: Describe how a Disposable is created ig.**

1 module AgdaInteraction where	1 module AgdaCommand where 
2 record t : Set where	2 data t : Set where
3 constructor iotcm	3 load : t
4 field	4 give refine-or-intro
5 file : TextDocument.t	5 : Bool → InteractionPoint.t → t
6 command : AgdaCommand.t	6 {- Other commands are elided -}

Listing 6: Definitions of `AgdaInteraction.t` and `AgdaCommand.t`. These are used to encode interactions that need to be sent to Agda.

The `AgdaInteraction.t` and `AgdaCommand.t` data types mirror the Haskell definitions in the Agda compiler. These data types describe what commands are available to run and what information is required to run them. They have serialisation functions that generate the stringified Haskell expressions that evaluate to the Haskell version of the data types. The Agda interaction mode uses some `Read` instances to read the string into the data type.

An `AgdaInteraction.t` is created through the use of two functions: `from-AgdaCommand` or `under-cursor-command`. `from-AgdaCommand` creates an interaction from a complete `AgdaCommand.t`, and `under-cursor-command` is a specialisation of `from-AgdaCommand` that uses the model and the cursor position to determine the goal under the cursor and pass that to a partially applied `AgdaCommand.t`.

3.3. Receiving and handling responses

Once we have an `AgdaInteraction.t`, Avea sends it to the Agda process. Agda responds with many JSON messages delimited by newlines per interaction. The child process data handler receives buffers of data from the standard output. These buffers may contain parts of the responses for an interaction, or multiple interactions even. Additionally, the last response might not even be complete too. The data handler therefore has the task to collect as many complete responses, and handle them.

Each handler may cause edits to the buffer or other asynchronous actions. However, if the handlers are invoked immediately from the handler, then there is no guarantee that the actions are executed in a deterministic order. This is because the data handler from Node.JS's child process library uses a callback function from which the promises of the `activate` function cannot be bound with. So the promises are immediately started and act as separate threads, which can introduce a number of concurrency problems.

To mitigate concurrency problems, Avea uses a concurrent queue which has a limit set to 1 to make sure only one response is handled at a time. This prevents edits from being made in the wrong order, or while other edits are being made, causing VS Code to throw an error.

The output of the data handler is a list of complete Agda responses as a list of parsed JSON objects. These objects are then parsed once again into more specific data types per response type. Each JSON object holds a `kind` property that can be used to distinguish the objects and drive an efficient parser from the JSON objects to Agda data types.

Parsing objects from JSON

To efficiently parse strings to JSON objects, and JSON objects to Agda data types, Avea makes use of the builtin support for JSON in Javascript. In Javascript, JSON objects can be parsed to Javascript objects using the `JSON.parse` function, which takes in a JSON string and produces Javascript objects, arrays and primitives corresponding to the JSON structure. In the Agda code, there is a JSON data type that has constructors for each type of JSON object. The FFI makes sure that the Agda data type constructors represent the primitive JSON objects returned by `JSON.parse`. Listing 7a describes the

JSON primitives, whereas Listing 7b and Listing 7c implement the bridge between the Agda data type and the Javascript objects.

```
1 data JSON : Set where
2   j-null : JSON
3   j-string : string → JSON
4   j-bool : boolean → JSON
5   j-number : number → JSON
6   j-array : List JSON → JSON
7   j-object : StringMap.t JSON
8           → JSON
```

Listing 7a: Agda data type with a constructor for each JSON primitive.

```
1 {-# COMPILE JS JSON = ((x, v) =>
2   x == null          ? v["j-null"]()
3   : typeof x == "string" ? v["j-string"](x)
4   : typeof x == "boolean" ? v["j-bool"](x)
5   : typeof x == "number" ? v["j-number"](x)
6   : Array.isArray(x)    ? v["j-array"](x)
7                       : v["j-object"](x))
8   #-}
```

Listing 7b: FFI code for the pattern matching on the data type.

```
1 {-# COMPILE JS j-null = null #-}
2 {-# COMPILE JS j-string = s => String(s) #-}
3 {-# COMPILE JS j-bool = b => Boolean(b) #-}
4 {-# COMPILE JS j-number = n => Number(n) #-}
5 {-# COMPILE JS j-array = l => [...l] #-}
6 {-# COMPILE JS j-object = kvs => kvs #-}
```

Listing 7c: FFI code to create a constructor of the JSON data type.

The benefit of using the FFI is that parsing JSON is also stack-safe. Implementing a string to JSON parser using recursion will quickly run into the stack depth limit, especially since some of the Agda responses can contain a sizable number of characters – e.g. the length of highlighting response grows linearly with the source file’s size. Javascript’s `JSON.parse` is stack-safe, and performs better, since its implementation is written in C++ using the NodeJS FFI⁶.

The parsing from JSON objects to Agda data types is done in a separate step. Avea contains a simple parser combinator library that

Handling parsed responses

After the data handler has collected a number of responses and parsed them to JSON, it remains to parse the JSON responses to specific data types which have more specific information about the response type. These data types can be passed to a handler which calls the relevant VS Code API. Listing 8 shows how the handlers are called from the JSON parsers: each decoder function parses their corresponding kind of response and passes a data type to the handler function.

```
1 handle-agda-message : (AgdaInteraction.t → IO T) → Ref.t Model → Model
2                       → Decoder (IO Model)
3 handle-agda-message send-command model-ref model =
4   (| (handle-highlighting-info model) highlighting-info-decoder
5   | (handle-interaction-points model) interaction-points-decoder
6   | (handle-give-action send-command model) give-action-decoder
7   | ...
8   |)
```

Listing 8: JSON decoder for the responses Agda sends

⁶<https://github.com/v8/v8/blob/main/src/json/json-parser.cc>

This section will continue to describe how the handler for three response types work to illustrate what response handlers do, and the constraints they are subject to.

Handling highlighting information

The highlighting information contains semantic tokens and a flag that indicates if the tokens replace the previous information or not. The tokens are sorted according to their positions in the source file, and then set in the internal cache. The internal cache keeps track of the latest tokens and interaction points per file. The sorting is important for the change handler to be able to efficiently search through the tokens, as will be described in Section 3.4.

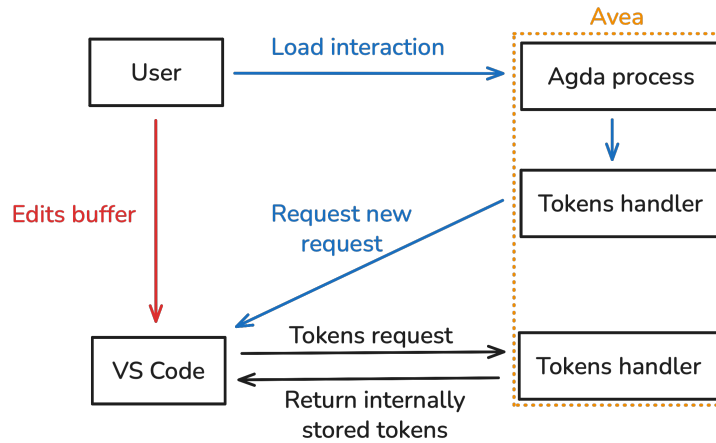


Figure 13: Diagram describing the handling of highlighting tokens. The user can request new highlighting either by making edits in the buffer (red path) or by invoking the load interaction (blue path). Avea can supply highlighting tokens on VS Code’s request via the tokens handler.

Once, the tokens are updated in the internal cache, the handler will send a notification to VS Code for it to retrieve new tokens. VS Code will automatically request new semantic tokens after a small batch of edits are made, but such a tokens request can also be requested by the extension. An example of a situation when this is useful, is when the user loads a file that has not been loaded before.

When VS Code requests new semantic tokens, the tokens handler registered in the `activate` function will fire. This handler simply looks for the saved tokens in the internal cache, and return semantic tokens for the tokens, and editor highlighting for errors, warnings, and interaction points. Errors and warnings are typically displayed as a change in background behind the code it applies to. For this reason, Avea has to apply editor highlighting separately for those tokens, because semantic tokens cannot be used to control the background of a piece of text.

Handling updates to interaction points

Agda sends this responds to let Avea know where the latest interactions points are. Each interaction point has an id and source locations of their start and end. The id can be used to refer to a specific interaction point in subsequent interactions. The source locations are used to highlight the interaction point, and to retrieve the expression contained within. Avea keeps track of interaction points in its own local cache, and the main purpose for the interaction points handler is to update this cache with the latest information from Agda.

These interaction points can come in two kinds:

- Question marks are one character wide interaction points. The user can place a question mark in an expression to initiate a hole. While responding to any interactions involving type-checking, the Agda compiler will parse the question marks and return them as interactions points in this response. It is the extension’s responsibility to expand the question mark into hole, as illustrated in Figure 14.

It is therefore convenient that Agda includes question marks in the interaction points response, since these can be filtered out and expanded in the buffer.



Figure 14: Illustration of question mark expansion: when the user types a question mark in an expression (left image), Agda will signify using an interaction points response where the question marks are, so that Avea can expand it in the buffer to a hole (right image).

- Holes are interaction points that are at least four characters wide. Holes contain an expression, which is delimited by goal markers `{!` and `!}`. The expression in the hole need not be a valid Agda expression, both in terms of syntax or in terms of type. Other interactions can operate on the hole, by inspecting the inner expression. These interactions require the latest content of the hole, since Agda does not actively keep track of a hole's contents.

The interaction points response is included as a response to any interaction that can change the size or number of interactions points. Common interactions that returns interactions points are give (`ctrl+c ctrl+spc`), refine (`ctrl+c ctrl+r`) and load (`ctrl+c ctrl+l`). However, the interaction points each interaction returns can vary. In particular, newly introduced question marks by the refine command have invalid source locations (0-wide), and therefore have to be discarded.

After discarding, the question marks are filtered out and are expanded. Expansion involves updating the local cache that saves the location of the start and end markers of holes, as well as replacing the question mark with goal markers (`{! | !}`) in the buffer, and horizontally shifting the position any interactions points on the same line in the buffer.

Handling give actions

This response is specific to the give and refine interactions. It contains a give result – which is either a new expression, or a command to use the expression currently contained in the hole as the replacement: either the expression in the goal is parenthesised and the goal markers are removed, or the expression is let as-is and the goal markers are removed. See Listing 9 for the definition in Avea.

```
1 module GiveResult where
2   data t : Set where
3     parens no-parens : t
4     str : String → t
```

Listing 9: Definition of the `GiveResult.t` data type, which describes how a give action should replace its associated hole.

This handler does the following steps:

1. Depending on the give result, the handler queries the VS Code buffer to retrieve the contents of the hole. The give action response contains the id of the interaction point in question, which can be used to search the latest source location in the extension's internal cache.
2. The replacement expression may also contain new interaction points, which the handler needs to expand. Agda gives the new interaction points in a separate in interaction points response, which is not accessible from the handler of the give action response. However, even if it was available in the give action response, Agda returns incomplete information about the newly inserted question marks.

This requires Avea to do a manual search through the give result for expandable question marks, and replace those in the expression string to empty holes. The algorithm for this makes use of regex search to categorise parts of an expression within which a question mark should not be expanded. In particular, question marks that are part of identifiers, appear within a string, or appear within comments should not be expanded, as illustrated in Figure 15.

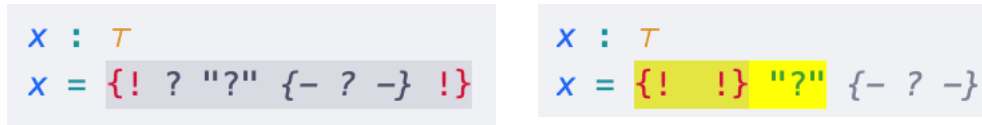


Figure 15: Expansion of question marks after a refine interaction. The left image is before the refinement, the right image shows that only the first question mark is expanded.

3. All expansions are collected into a list of edits that can be made to the VS Code buffer. Additionally, if any expansion have been performed, the cursor is moved to the first new hole, to improve developer experience.
4. A load interaction is issued, so that the newly created holes will correctly be registered by that interaction's interaction points response. This also means that the internal cache need not be updated since the load interaction will elicit an interaction points response.

3.4. Reacting to changes from the user

As mentioned earlier, Agda responds with a list of semantic tokens as a response to some interactions. A semantic token consists of a start and end location, as well as, what Agda calls *primary* and *secondary aspects*. Primary aspects are used to display the function of a token to the user. For example, variables with function types are coloured blue, whereas variables of type Set are coloured yellow. Secondary aspects contain additional information about tokens, for example, whether the token's usage causes an error or warning, or whether it is part of a pattern that is non-exhaustive.

The semantic tokens provider is queried by VS Code whenever the user has edited a buffer. These edits are not communicated to the semantic tokens handler: it needs external listeners to receive updates. The reason these edits are needed in the first place is that Avea saves the locations of all tokens and interaction points. However, when the user edits the buffer these locations might change. Sending the same locations for all tokens will result in pieces of code being highlighted with wrong or incomplete information, for example, identifiers with two colours or missing colours, see Figure 16.



Figure 16: Illustration of syntax highlighting in Avea: the left image is a loaded file. The middle images shows an edited buffer when no shifting is performed, whereas the right image shows the same edited file with shifting enabled.

Initially Avea, stored the tokens as Agda gives them. This means that no processing needs to be done when receiving the tokens, but shifting the tokens requires iterating over the entire list. Typically, a buffer is edited more times than it is loaded, so this approach can cause visual slow downs if the buffer has a large number of tokens. Another approach, which Avea uses now, is to store the tokens in sorted order. This means that the received tokens need to be sorted in $\mathcal{O}(n \log(n))$ time. The handler does a binary search to find the first token affected and continue shifting the tokens until a token on a new line. The key observation is that only tokens on the same line after the edit need to be shifted.

3.5. Input mode

There is one component of the extension that operates completely separately from the compiler: the input mode. Technically, the input mode could even run as its own extension, since it shares no resources with the rest of Avea: it does not require access to Avea internal caches. **TODO: This feels incomplete, but idk what else to put here yet.**

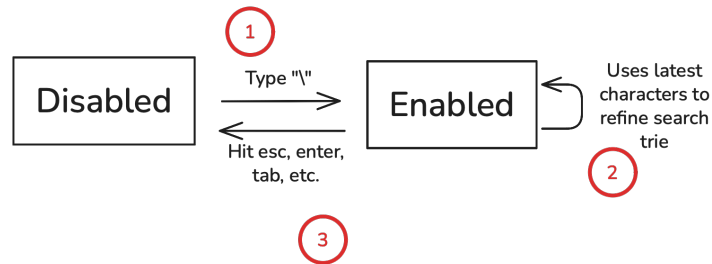


Figure 17:

The input works according to a simple state machine as described in Figure 17. Though each transition has some side effects:

1. By default the input mode is disabled. There is one handler that listens for the “\” character, which enables input mode.
2. Any subsequent characters are used to traverse a trie in a zipper-style. The zipper can be used because a user can type another character to traverse deeper into the trie, but the user can also hit backspace to go up in the trie. A zipper is a perfect structure to perform these traversals on in a purely functional manner. **TODO: Maybe add a reference to a paper introducing tries?**
3. If the user types a character that is not a child of the focus node, or the user hits the escape key, then input mode exits and the character is inserted into the buffer.

4. Evaluation

4.1. Importing Javascript libraries

The extension uses some external libraries that need to be imported in the Javascript output, in order to function. Usually in Javascript, the `import` statement is used to import libraries. However, the latest release of the Agda compiler does not provide a way to import Javascript libraries. Instead the extension relies on a separate Javascript file to set the dependencies in a global variable available across all modules, and then call the extension's main function, as shown in Listing 10.

```
1 import Main from "./jAgda.Avea.Extension.mjs";
2 import * as vscode from "vscode";
3
4 export function activate() {
5   Object.assignProperty(globalThis, "AveaImports", {
6     value: { vscode, /* and some other libraries */ }
7   });
8   Main(); // Call the main function from the extension manually
9 }
```

Listing 10: A Javascript file importing libraries and making them globally available before running the Agda code.

To illustrate, the code in Listing 11 below throws an error at runtime. This is because the evaluation of `imported-string` for the value of `s` happens before the `activate` function is called. This means that `AveaImports` is not defined when `imported-string` is evaluated, and therefore `someString` cannot be accessed, throwing a runtime error.

```
1 postulate imported-string : String
2 {-# COMPILER JS imported-string = AveaImports.someString #-}
3 s = imported-string <> "!"
```

Listing 11: Illustration of using `AveaImports` at the top-level, which causes runtime errors.

This issue can be solved by wrapping `imported-string` into `IO` as illustrated in Listing 12. This delays the evaluation of `AveaImports` until it can be used, which, in the extension, is guaranteed to be after `AveaImports` is defined. Even though this approach works, it only allows top-level definitions to be used in effectful functions, even if the values themselves might be pure. This leads to unnecessary colouring of functions to be `IO`, or passing the top-level definitions as parameters to functions that need them, undermining the use of the top-level definition.

```
1 postulate imported-string : IO String
2 {-# COMPILER JS imported-string = async () => AveaImports.someString #-}
3 s = fmap (_<> "!") imported-string
```

Listing 12: Using `AveaImports` at the top-level without causing runtime errors by delaying the access to `someString` through the use of `IO`.

A similar issue has already been solved for the Haskell backend through the `FOREIGN GHC` pragma. The `FOREIGN GHC` pragma allows the user to insert arbitrary code at the top of the output files. However, the Javascript version of this pragma, `FOREIGN JS`, is still unimplemented at the time of writing. Implementing the `FOREIGN JS` pragma requires very little changes to the Agda compiler, and

we created a fork of the Agda compiler that includes support for the pragma⁷. An example use of the pragma is shown in Listing 13.

```
1 {-# FOREIGN JS import { someString } from "library" #-}
2 postulate imported-string : String
3 {-# COMPILE JS imported-string = someString #-}
4 s = imported-string <> "!"
```

Agda

Listing 13: Using the FOREIGN JS and COMPILE JS pragmas to import a variable from a Javascript library.

The FOREIGN JS pragma allows the modules to import their Javascript dependencies without requiring a separate file to bind them to the global scope, and allows these dependencies to be used at the top-level without requiring IO. This is because it allows the import of someString in the same file where the imported-string variable is defined. Given that the FOREIGN JS is placed before imported-string is used, someString can be correctly resolved, and the code runs without any errors. Unfortunately, since the implementation of the FOREIGN JS pragma has not been finalised and released yet, Avea still makes use of the imports object approach.

4.2. Handling asynchronicity

By default Agda expects IO operations to use continuation-passing style, though the builtin module does not provide the monadic operations: pure and `_>=>_`. When compiling, the Agda compiler looks for a main function, and if it exists, it inserts `main(() => {})` at the end of the file. The standard library author, therefore, has a choice how to implement these functions as long as calling the function starts the IO operation.

Initially, the extension made use of continuation-passing style for IO (provided by the Iepje library⁸). However, Visual Studio Code exposes some functions that return promises: a type that modern Javascript uses to handle asynchronous computations. A promise represents a computation that has already started and will return a value in the future, an example of which can be found in Listing 14.

```
1 const delay = (time, f) => new Promise((resolve, reject) =>
2   setTimeout(
3     () => resolve(f()),
4     time
5   ));
6 delay(1000, () => 10).then(n => console.log(n)); // Prints 10 after 1 second
```

JavaScript

Listing 14: Example function that uses promises that delay a computation and waits for the computation to finish to print the result.

To be able to use these functions from the Visual Studio Code API, the extension requires a different definition for IO, where instead of compiling to continuation passing style, we use lazy promises. Using just promises for the IO monad – with the new constructor as pure, and `.then` for `_>=>_` – means that any term of type IO A has already started executing. Since the extension makes abundant use of “lazy” IO, where storing the IO action does not mean it has already started executing, we have decided to define IO as functions of Typescript type `<T>() => Promise<T>`.

However, in order to avoid stack overflows when, for example, traversing large lists with the IO monad, we need to make use of the syntax sugar that Javascript has for promises: `async/await`. As seen in

⁷<https://github.com/agda/agda/issues/8429>

⁸<https://github.com/lawcho/iepje>

Listing 15, an implementation with `async/await` is a little more verbose, but reduces the stack depth of nested IO operations compared to using `.then` directly – which adds a stack frame every time it is called.

<pre> 1 ma => f => async () => { 2 const a = await ma(); 3 return await f(a)(); 4 }</pre>	<pre> 1 ma => f => () => 2 ma().then(a => f(a)())</pre> JavaScript
--	---

Listing 15: Juxtaposition of an implementation of `_>=_` with stack depth linear to the nesting depth of the `_>=_` operations (left), and an implementation with stack depth linear to the number of evaluated `_>=_` calls (right).

4.3. Mitigating stack overflows

The Agda compiler does not support tail-call optimisation. Since recursion is the only method of traversing a data structure, this means that the stack size determines how deep we can traverse a data structure in one pass. For the implementation of the extension this poses a serious problem, since the responses from Agda containing the highlighting tokens grow linearly with the size of the source file that is being checked. For relatively small source files, we already run into the stack limit.

To mitigate this, we need to use the JavaScript ffi to define “stack-safe” functions that can traverse large data structures without increasing the stack depth per element. Agda allows us to provide a definition that is used by the elaborator, and another separate one that is used in the compiled output. This means we retain the ability to reason over the functions, as well as removing the worries about the stack depth.

<pre> 1 reduce-right : B → (B → A → B) → List A → B 2 reduce-right b f [] = b 3 reduce-right b f (x :: xs) = f (foldr b f xs) x 4 5 {-# COMPILER JS reduce-right = a => A => b => B => b => f => xs => 6 xs.reduceRight((ac, e) => f(ac)(e), b) #-}</pre>	Agda
---	--

Listing 16: The implementation of the `reduce-right` function both for Agda’s elaborator and for the compiled output. The Javascript definition offers stack-safety.

While this method does resolve stack-safety issues, we do have another problem. Agda takes the `COMPILER JS` pragma and inserts it unchecked in the compiled output. Therefore, there is no guarantee that the definition in the pragma adheres to the type declaration, and that the pragma definition has the same semantics as the Agda definition.

A pragmatic solution to this “consistency checking” problem is to create a separate program that does the following:

1. For each test case, the program requests a fully qualified function name, and test inputs for each relevant parameter. The program will compile the library and find the correct Javascript definition in the compiled output.

<pre> 1 run_consistency_tests 2 name: "Data.List.map" 3 inputs: cartesian_product test_functions, test_lists 4 erased_params: 4 5 meta_information: "{A = Nat} {B = Nat}"</pre>	CoffeeScript
---	--

2. The given inputs are applied to the function, taking the erased parameters into account, and then they are serialised together with the output for the next step.
3. The outputs from the compiled JavaScript code are inserted into a temporary Agda module that prompts the elaborator whether the the same function call gives the same output. In particular, the tester creates Agda source files with the structure described in Listing 18.

```
1 module Test where
2 -- Import the module containing the function we want to check.
3 open import Data.List
4 open import Agda.Builtin.Equality
5 -- For each test case, there is an assertion that the Agda definition
6 -- normalises to the same output as the JavaScript definition did.
7 _ : map (λ x → x + 1) (1 :: 2 :: 3 :: []) ≡ (2 :: 3 :: 4 :: [])
8 _ = refl
```

Listing 18: Agda file for checking consistency once the output of the JS definitions is collected.

If this file type checks, it means that for each test case, the Agda definition is return the same results as the Javascript definition. Otherwise there might be issues in i.a. the number of parameters, different semantics, or type errors in the Javascript definition.

With well-chosen test cases, there is a reasonable degree of consistency between the Agda and JS definitions. By transitivity, the JS compiled output has a similar degree of correctness if the Agda definition is verified. With this method, we have a way to interact with the Javascript FFI in such a way that we have some degree of quality assurance.

There are some limitations to this tool, especially when testing functions operating on more complex data types, or function using type classes. Data types and type classes are compiled to more complex objects, and may make use of external functions whose structure and generated names are quite hard to predict.

5. Adding extensibility

In this chapter, we will look into making Avea extensible. For this we have decided to heavily depend by previous work on editor reflection by Korkut and Christiansen [13]. They introduced the concept of editor reflection in Idris1 – a language with a similar metaprogramming library, and a similar compiler structure as Agda.

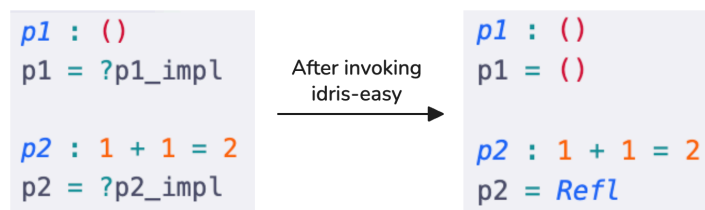


Figure 18: Example use of the `idris-easy` editor action defined below. The action can be executed on a hole, and it is filled with the returned expression from the meta-program.

In brief, editor reflection allows users of Idris1 to write meta-programs that can be executed from the editor. Listing 19 lists the code used to implement the example editor action taken from [13], whose functionality is shown in Figure 18. The meta-program in Listing 19a has a goal name parameter and attempts to solve the goal by inspecting its type. If the type of the goal is unit, it will return unit (line 6), if the goal type is an equality type and the types and values are convertible, then the reflexivity constructor is returned (line 10), otherwise the goal is not trivially solvable and an error is returned. This rudimentary solver is invoked by the editor using the Emacs Lisp support code (also called extension code) in Listing 19b. This code defines the editor action and what arguments (line 6) to send to the meta-program.

```

1  %editor                                     Idris1
2  easy : TTName -> Elab TT
3  easy n = do
4    ty <- getType n
5    case ty of
6      `(() : Type) => pure (() : ())
7      `(=) {A=~a} {B=~b} ~x ~y => do
8        converts a b
9        converts x y
10       pure `(Refl {A=~a} {X=~x})
11    _ => fail
12    [TextPart "Cannot solve"]
    
```

Listing 19a: Idris1 meta-program that tries to solve a trivial goal.

```

1  (defun idris-easy ()                               Emacs
2    "Example editor action"
3    (interactive)
4    (idris-elab-hole-arg
5     "easy" (list
6             (idris-name-at-point))))
    
```

Listing 19b: Emacs Lisp support code to call the meta-program on the left from within the Idris1 editor.

While the design by Korkut and Christiansen makes for an extensible editor tooling, users of Idris1 still have to write Emacs Lisp code to create an interface between the editor action and the editor itself. This has two drawbacks:

1. There is a potential source of bugs where the editor action returns an unexpected data type from the Emacs Lisp support code's perspective, or the support code sends an unexpected data type in the editor action's perspective. There is no way currently to constrain both ends to use the same data types.

2. Users need to learn the Emacs Lisp DSL. This might not be a big problem for smaller editor actions, however, more advanced editor actions might require more processing on the editor side, which requires more advanced knowledge of Emacs Lisp.

These drawbacks can be solved by allowing the support code to be written in the same language as the editor actions. Since Avea is written in Agda already, and Agda has similar meta-programming capabilities, Agda is a perfect candidate for this. The example from Listing 19 would be translated to the Agda program in Listing 20.

```

1  easy : Term → TC Term
2  easy hole = inferType hole >=> normalise >=> λ where
3    (def (quote τ) []) → pure (quoteTerm tt)
4    (def
5      (quote _≡_)
6      ((arg _ l) :: (arg _ A) :: (arg _ x) :: (arg _ y) :: [])) → do
7      l' ← unquoteTC {A = Level} l ; A' ← unquoteTC {A = Set l'} A
8      unify x y ; x' ← unquoteTC {A = A'} x
9      pure (quoteTerm (refl {A = A'} {x = x'}))
10   _ → typeError (strErr "Cannot solve" :: [])
11
12 agda-easy : EditorAction.t τ
13 agda-easy = do
14   term ← hole-under-cursor >=> get-hole-content
15   new-term ← call-editor-action easy [ term ]
16   hole-under-cursor >=> fill-hole-with new-term

```

Listing 20:

The first drawback is solved, because Agda data types can be shared between the editor action and support code, which can act as a checked contract between the two parties.

Even though the second drawback would also be solved, it is important to note that the extension code is still executed in a different environment as the editor action. Figure 19 describes all environments the different pieces of Agda code can be executed in. It shows that extension code (yellow box) can call editor actions (blue box) through Avea's API (green box), however editor actions cannot call back into the extension code. Extension code can also run regular meta-programs (black box), which will be executed at the compile time of the extension code. There is a subset of Agda code that can be used in any environment (red box). The constraints for this is that this code can only make use of data types defined within Agda itself, or of builtin data types and primitive functions. Logically, this excludes any use of runtime-specific code, i.e. code using postulates and `COMPILE` pragmas.

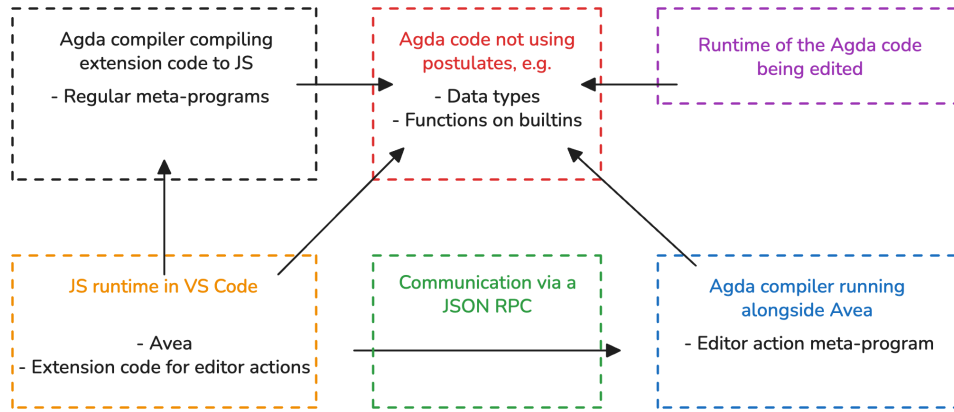


Figure 19: A diagram mapping out the different environments Agda code can be executed in. The arrows symbolise an environment calling Agda code written for another environment.

To implement an editor with editor reflection fully in Agda, we can extend Avea and the Agda compiler to support editor reflection, and for this we identify the following stages of development. These stages are ordered such that each stage adds new functionality on top of the previous stages. Even though the implementation of these stages goes beyond the scope of this thesis, we will still describe a detailed design for each stage.

- In Section 5.1, we will introduce a basic embedded domain specific language (EDSL) in Avea for listening for events in VS Code and handling them, and adding new edit-time functionality extensively without making changes to the Agda language or the Agda compiler. We will call these **extensions** in Avea. The new functionality of this system will be rather limited: this API would allow people to write snippets or automate a sequence of hole commands for example in an extensible manner.
- In Section 5.2, we will discuss being able to call meta-programs, which we call **editor actions**, in the Agda compiler. This involves adding a new command in Agda’s interaction mode, which takes the name of an editor action and executes the meta-program associated with the editor action with any arguments provided in the command and returns its result. The implementation of this is mostly based on the work of Korkut and Christiansen, but applied on the Agda compiler.

Adding reflection to the current version of the Agda compiler allows for non-syntactical expression manipulation in holes. For example, simplification of programs written in an EDSL, customised code snippets (e.g. generating boilerplate for type class instances), and proof search actions.

- Finally, in Section 5.3, we will discuss changing Agda’s meta-language to provide source locations: A design for this includes two levels of meta-language similar to Idris1: A Raw language with source locations and full range over all language constructs in the surface language, and a Term language which works as the current Term meta-language in Agda. There are elaboration functions of type `Raw -> Term`.

This allows syntactic changes and more precise edits in the buffer. This enables use cases such as lintings and warnings, which can be executed on every load of a file, and display squiggles in the buffer at exact locations. The VS Code allows squiggles to also includes fixes, which can trigger yet another edit-time function which can replace the source range of the expression with a linter warning or whatever with a suggested fix.

In the current Term language, Agda does not distinguish between many surface level constructs. This means that syntactically different but semantically equivalent notations cannot be generated or inspected. The two level meta-language also allows for style checkers and style changes, because the Raw language allows reasoning about syntactic constructs. This allows actions such as “convert

let to where”, which converts two ways of bindings variables. This action can also make use other metaprogramming features to determine the scope where to insert the let or where block in.

5.1. Start of the Avea extension EDSL

The starting point for an extension in Avea is a special file that contains all hooks and extension actions. Accompanying this file is a .agda-lib file that can be used to declare dependencies upon which the extension actions can make use of.

```
1 module Avea.Extensions.Hooks where
2
3 registrations : List ExtensionRegistration.t
4 registrations = on-hook load ... :: on-cmd ... :: ... :: []
```

Listing 21: Startup file for Avea extensions. The registrations variable is a list that says when a function should be called after what event.

The hooks are registered by calling constructors of the `ExtensionRegistration.t` type. The hooks are defined in Avea, and listen for events from a number of relevant VS Code features. For example, extension code can register a command in the command palette in VS Code or listen to lifecycle hooks that are executed after a particular event, e.g. on a successful load of a file.

```
1 module ExtensionRegistration where
2   data t : Set where
3     on-hook : AveaHook.t → ExtensionAction.t T → t
4     on-cmd : AveaCommand.t → ExtensionAction.t T → t
5     on-menu-button : MenuButton.t → ExtensionAction.t T → t
```

Listing 22: Definition of `ExtensionRegistration.t`, which is used to define hooks.

The extension actions that are executed when a hook triggers, are program written in the `ExtensionAction.t` monad. The `ExtensionAction.t` monad is a State monad where the state is Avea’s model. However, the monad does not allow any field in the model to be modified: changes or accesses must be done through the defined functions in the Avea library. This way extensions cannot interfere with Avea’s internal system, and potentially compromise its other functionality. So examples of functions that operate in this monad are described in Listing 23.

```
1 hole-under-cursor : ExtensionAction.t Hole.t
2 get-hole-content : Hole.t → ExtensionAction.t HoleTerm.t
3 get-hole-range : Hole.t → ExtensionAction.t Range.t
4 replace-hole-with : HoleTerm.t → Hole.t → ExtensionAction.t T
5 write-to-current-buffer : String → Range.t → ExtensionAction.t T
6 send-agda-interaction : AgdaInteraction.t → ExtensionAction.t AgdaResponse.t
```

Listing 23: Signatures of functions in the proposed Avea extension EDSL.

The implementation of extensions in Avea should not pose too many problems. It requires:

- Implementations of the function described in Listing 23. These functions already exist within the current implementation, but need to be adjusted to work within the `ExtensionAction.t` monad.
- Each hook should be triggered at the appropriate time. For registrations involving commands or buttons, they should first update the UI and let the callback call the `ExtensionAction.t`. Each life cycle hook just needs access to the registration list to call the functions subscribed to the hook in question.

- Adding a new “reload extensions” command in Avea, which recompiles the Agda project containing the registrations and loads the compiled Javascript code and updates the registrations in Avea’s model.

Use cases

The extension EDSL does not unlock very powerful edit actions, apart from some automations of existing sequences of Agda interactions. However, there are still some edit actions that were not possible to implement before, which are listed below. Additionally, this EDSL forms a solid base to integrate the next stages in.

- *Caching for proofs by reflection*: proof by reflection is a technique where functions mechanically generate proofs for a theorem [15]. Computing these proofs for large proof trees can be expensive, especially considering that these proofs may not change often. Therefore, it might be more efficient to inline the proof tree in the source to avoid doing the proof by reflection at elaboration-time. Inlining the proof tree could be implemented as an editor action in our extension.
- *Record and replay*: Listening to hooks of all interactions, it is possible to record how a definition is “created” by recording the use of Agda interactions. These can be saved and serialised, and later on replayed on another definition to recreate it, or exported for didactical purposes.

5.2. Edit-time actions in Agda

Instead of modifying the compiler to add editor features, Korkut and Christiansen [13] have implemented an extensible editor for Idris1 where editor commands can be defined within Idris1 itself using the metaprogramming library. The procedure they used to implement the editor extensions can be translated quite well to Agda. The procedure consists of defining the communication protocol between the interaction mode and Avea, designing a user-friendly API to call editor actions in a type-safe manner, and running editor actions in the compiler.

1. Defining the communication protocol

Editor actions are executed in the Agda compiler. However, they are triggered by the extension and receive arguments from there, and the result of the editor action also needs to be communicated back to the extension. To facilitate this communication, we can define two type classes: `Encodable` and `Editorable`, whose definitions can be found in Listing 24. The extension sends values to and from the compiler using JSON. The `Encodable` type class captures the (de)serialisability to and from a type to JSON, and `Editorable A B` combines two `Encodable` instances to capture convertibility of values of type `A` to `B` via JSON.

```

1 record Encodable (A : Set) (M : Set → Set) : Set₁ where field
2   to   : A → JSON
3   from : JSON → M A
4
5 record Editorable (A B : Set) : Set₁ where field
6   runtime : Encodable A Maybe
7   action   : Encodable B TC

```

 Agda

Listing 24: Possible definition of the `Editorable` type class for (de)serialising reflection types.

The reason why `Editorable` has two type parameters `A` and `B` is that the type of values on the runtime and the editor action might differ. Take lines 14-16 in Listing 20 as an example. Line 14 requests the hole under the cursor and if it exists, then request its contents. However, a hole’s contents might not resemble a valid Agda expression. Furthermore, parsing that expression in Avea requires embedding Agda’s parser pipeline in Avea, which is difficult to do since the Agda parser is written in another language than Avea.

For this reason, Avea keeps the hole's contents as a `String`, but it will be parsed to an Agda term in the editor action. To keep the API type-safe, the `Editable` instance takes two parameters: one for the representation in Avea, and one for in the editor action. Calling the editor action, will pass along the serialised data from `A` and the `Editable A B` instance that is used to do it. This means the editor action can deserialise the data to the correct type, and serialise it back correctly.

Even though, `Editable` takes a source and target type, they can be the same. This also simplifies the construction of an `Editable A A` instance, since there exists a simple translation from an `Encodable A Maybe` instance to an `Encodable A TC` instance. This is illustrated in Figure 20.

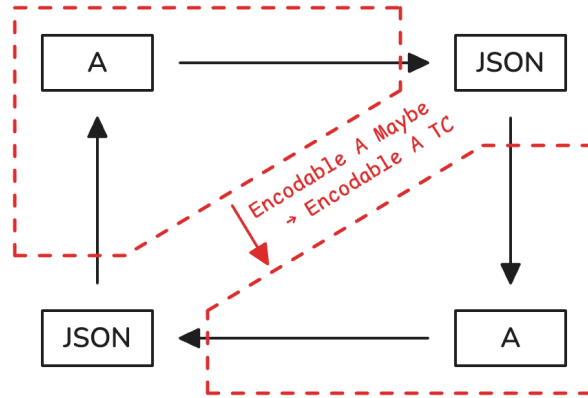


Figure 20: Diagram of an `Editable A A` instance: it consists of two `Encodable A M` instances (red boxes), which each contain two functions. Given an `Encodable A Maybe` instance, we can complete the square through a simple transformation.

The definitions of `Encodable A M` and `Editable A B` can be subject to some laws proving their correctness. The laws are shown in Figure 21. The red box shows an `Encodable` instance, where it should be that deserialisation after serialisation should not change the result. The same law holds for `Editable` (shown in the green box), however, the serialisation and deserialisation are given by the transfer functions. The transfer functions are the composition of the serialisation and deserialisation via JSON. This law makes sure that, even though the individual transfers via JSON are sound, the overall conversion to and from `A` and `B` is also sound.

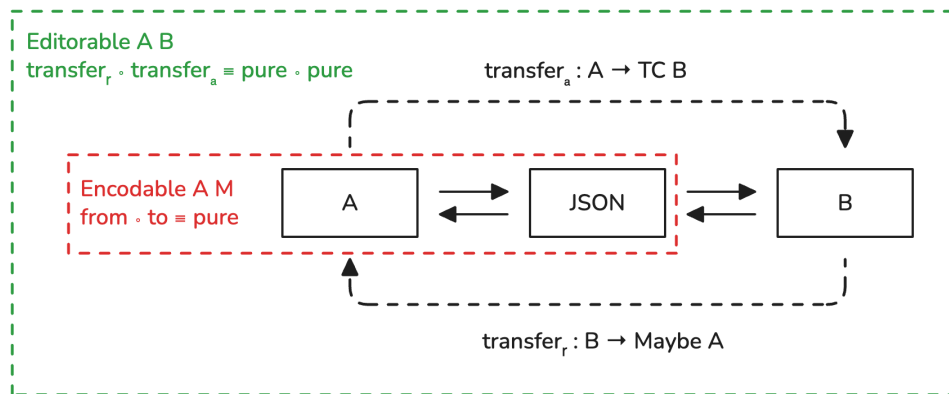


Figure 21: Visualisation of the laws proving correctness of the communication. In an `Encodable A M` instance, `from` and `to` should be inverses. The same applies for `transfera` and `transferr` in `Editable A B` instances.

2. Designing the API to call editor actions

An editor action consists of two parts: the (de)serialisation, and the actual metaprogramming logic. The `EditorAction.t` record contains the type of the metaprogram and the metaprogram augmented

with the (de)serialisation steps. Augment the meta-program in this way, and saving its original type separately, will make the implementation of executing editor actions in the Agda compiler easier (as discussed in stage 3). Using meta-programming, it is possible to automatically convert a metaprogram to an `EditorAction.t`. This process is encoded using the `editor-action` macro, whose signature can be found in Listing 25.

```

1 module EditorAction where
2   record t : Set1 where field
3     action-type : Set
4     action : List String → TC String
5
6 macro editor-action : (m : Term) (hole : Term) → TC  $\tau$ 

```

Listing 25: Definition for `EditorAction.t` and the `editor-action` macro, which converts regular meta-programs to `EditorAction.ts`.

The `editor-action` macro can infer the types for each parameter, and searches the correct `Encodable A TC` instances. Then it generates a new meta-program of type `List String → TC String`, that runs the deserialisation functions on the parameters, then runs the meta-program `m` with the deserialised arguments, and lastly, serialises the result of calling `m`.

Calling the interaction mode for an editor action can also be made easier through the use of meta-programming. The meta-program `call-editor-action` shown in Listing 26 takes the name of the editor-action that should be invoked and a list of arguments to pass to the editor action.

```

1 macro call-editor-action
2   : (action-name : Name) (args : List Term) (hole : Term) → TC  $\tau$ 

```

Listing 26: Type signature for the `call-editor-action` macro. This macro is responsible for calling an editor action from the extension code.

This macro does the following steps:

1. Firstly, it fetches the type of `action-name`'s type signature from the environment, which should be of type `EditorAction.t`. The parameters of the type saved within the record is unified with the types of the corresponding given argument. This check gives type-safety when calling the editor action.
2. For each parameter and the return type, the instances for `Editable` are searched and saved. These instances are used to generate the code to serialise each of the given arguments. If there exists no `Editable` instance for a type, then a type error is raised.
3. The meta-program generates code of type `ExtensionAction.t (Either String B)` for the following steps:
 1. Serialise the arguments using the found `Editable` instances for the parameters.
 2. The `action-name` and the serialised arguments are inserted into a `iotcm` call, which is sent to the Agda interaction mode handle that Avea keeps open.
 3. When Agda responds with the serialised result of the editor action, the result is deserialised using the `Editable` instance for the return type, and this deserialised result is returned. The editor action might also fail, either because the editor action threw a type error, or because deserialisation failed (e.g. when an expression in a hole is not valid). For this reason the generated code returns either the stringified error or the expected result.
4. The generated program is unified with `hole`. Using the macro allows for seamless integration with the `EditorAction.t` monad defined earlier. An example usage of the `call-editor-macro` can be found in Listing 27.

```

1 editor-action : EditorAction.t
2 editor-action term = editor-action ...
3
4 extension-code : EditorAction.t →
5 extension-code = do
6   term ← hole-under-cursor >=> get-hole-content
7   call-editor-action editor-action [ term ] >=> λ where
8     (left err) → display-error err
9     (right new-term) → hole-under-cursor >=> fill-hole-with new-term

```

Listing 27: Example usage of the `call-editor-action` macro from the `EditorAction.t` monad, combining operators from Section 5.1 with editor reflection.

One last ingredient to make the example in Listing 27 work is the serialisation of the term retrieved from the hole, and the term inserted into the hole. Avea cannot parse the expression in a hole (which might not even be a valid expression at all), and so it will return a string. However, the `editor-action` expects a `Term`. Instead of returning the hole contents as a `String`, Avea can return another data type `HoleTerm.t` containing a `String` with a custom `Editable` instance from `HoleTerm.ts` to `Terms`.

```

1 module HoleTerm where
2   record t : Set where field term : String
3
4   editable : Editable HoleTerm Term
5   editable = record
6     { runtime = -- Encodable instance for HoleTerm
7       ; action = Encodable-Term }
8
9   get-hole-content : Hole.t → EditorAction.t HoleTerm.t

```

Listing 28: Definition for `HoleTerm.t` which is a wrapper data type for `String` representing terms inside of holes. The wrapper type allows a custom `Editable` instance to `Terms` from Agda’s reflection library.

3. Running editor actions in the compiler

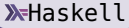
Running the editor actions requires two changes in the compiler: the extension needs to be able to call an action using the a new endpoint in the interaction mode, and the editor action needs to be able to de-serialise its arguments and serialise its return value.

The new endpoint in the interaction mode is relatively simple to add: the Agda compiler has an `Interaction` data type, where each constructor is an endpoint. It suffices to add a new constructor to this data type and implement the relevant `Read` instances. Every `Interaction` already receives information about the file it should operate on, so this need not be included in the parameters of the new constructor. What remains are parameters for the name of the editor action, the name of the `Editable` instance and the serialised arguments. The new constructor definition can be found in Listing 29.


```

1 data Interaction'
2   = ...
3   | Cmd_editor_action
4     String -- The name of the editor action
5     [String] -- List of serialised arguments

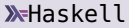
```

Haskell

```

1 data Response_boot tcErr tcWarning warningsAndNonFatalErrors
2   = ...
3   | Resp_EditorActionResult String -- Stringified result of the editor action

```

Haskell

Listing 29: Definition of the new interaction constructor for calling an editor function via Agda’s interaction mode (top), and the response constructor for receiving its result (bottom).

The semantics for the editor action endpoint have to be given in the `interpret` function in the compiler’s `Agda.Interaction.InteractionTop` module. Firstly, this function should look up the editor action by the provided name, where the name should correspond to a value of type `EditorAction.t` accessible in the top-level of the module the interaction operates on. The action property in the `EditorAction.t` record is a meta-program that accepts a list of serialised arguments, parses them and then executes the action itself. This meta-program also serialises the result of the action, which is ultimately returned. This means that, after looking up the value of the name, the endpoint only needs to call the meta-program with the serialised arguments, and wrap its return value in a `Resp_EditorActionResult` constructor to return to Avea.

Referencing the parsers in the meta-program itself allows the implementation of the endpoint to be quite simple, however it also has a limitation. The type of each parameter must be known at the definition of the meta-program. This means that meta-programs with dependent types, and by extension, polymorphism cannot be called using the approach described above.

With the implementation of `interpret` in place, it remains to implement `Encodable` instances for the data types in the reflection library. Since the editor actions operate on the types in the reflection library and use its functions, it is therefore necessary to be able to deserialise and serialise values of these types. As remarked by Korkut and Christiansen, the extension has access to stringified concrete syntax, that is, the language the users write, whereas the editor action operates on abstract syntax. To convert stringified concrete syntax to abstract syntax, it needs to be parsed, scope-, and type-checked. The reflection API does not expose primitives for parsing or scope-checking, so it is not possible to write instances for `Encodable` for some types in the reflection library. Instead, it is possible to expose new primitives that can be used to define such an instance. Their signatures are described in Listing 30.

```

1  data HasPrim : Set → Set where
2    has-Term : HasPrim Term
3    has-Definition : HasPrim Definition
4    -- Other types elided
5
6  primitive
7    primFromEditor : HasPrim A → JSON → TC A
8    primToEditor   : HasPrim A → A → TC JSON
9
10 instance
11   Term-Encodable : Encodable Term TC
12   Term-Encodable = record
13     { from = primFromEditor has-Term
14       ; to   = primToEditor has-Term }

```

Listing 30: Signatures for the from and to functions for the builtin reflection data types.

The new primitives `primFromEditor` and `primToEditor` also require an implementation in the Agda compiler. The `primFromEditor` function should, given a JSON object, check the tag if it matches with the expected type, and then convert the data into the data types used in the elaborator. The `primToEditor` does the opposite, given some type, it will delaborate and serialise the value into a JSON object with the correct tag.

Use cases

The editor reflection system as we have described it above allows for more powerful edit-time features. In particular, users can now combine the functionality of Avea’s extension EDSL and the functionality of Agda’s reflection API to write new tooling. This means that editor reflection allows users to transform expressions in holes or fill holes using a meta-program. Some examples include:

- *Hoogle-like lemma search*: this feature requests all names available in the current scope of the cursor and tries to unify them with a given type (or the requested type of the hole). This feature needs to communicate a list of types back to the extension, which is then displayed in the display buffer.
- *Re-implementation of some existing interactions*: With the ability to inspect the local context, it is possible to replace interactions such as `give (C-c C-SPC)`, `refine (C-c C-r)`, `goal environment (C-c C-.)`, `infer (C-c C-d)` and `normalise (C-c C-n)`. These interactions can be implemented using the reflection API, and therefore their implementation can be moved from being in the compiler to being editor actions in a standard library.
- *Custom automatic proof search*: Kokke and Swierstra [16] have implemented an extensible proof search mechanism using just Agda’s reflection library. Editor reflection allows users to call this proof search library at edit-time, which has a number of advantages, as remarked by the authors in the discussion. Firstly, the proof search does not need to run every time the module is compiled, since its resulting program can be spliced into the source code of the program. Secondly, the resulting program can also contain new holes, which opens the possibility for the proof search mechanism to return incomplete terms.

5.3. Changing Agda’s meta-language

TODO: Describe the need for control over the surface language in the reflection library. Additionally, mention Korkut’s future work

TODO: Describe the need for source locations in the surface language

With a change of reflection library, it is possible to unlock a lot more potential of editor reflection. In particular, it would be possible to be more precise about the location of where edits should be applied or where warning squiggles should appear. This is the driving mechanism behind the following examples:

- *Add Clause action*: Korkut and Christiansen implemented an “Add Clause” action, which, given a signature of a function, provides an initial skeleton for the definition of that function. The same would be possible with the change in the meta-language, now that the source location of the signature can be used to determine the insertion point for the definition.

```
1 append : List A → List A → List A    -- Given signature
2 append a b = {! !}                    -- Definition provided by the extension
```

Listing 31: Example of the code produced by the *Add Clause* action, which could be implemented with the proposed editor reflecton system and meta-language upgrades.

- *Unused variable linter*: This is a feature that many more mainstream languages have, where unused variables are greyed out or marked as a warning for being redundant. The editor action part of the linter can traverse the structure of the program, and keep track of usage per binder, when finally it can return the locations of the unused binders. The extension code can trigger on every load, where it will call the editor action, and put squiggles under every returned source location with a hover information message.
- *Style checkers*: Taking the idea of the previous linter a step further, style checkers analyse the structure of the code for general *anti-patterns*, that is, obvious structures that can be improved. For example, a programming team might prefer one particular way of pattern matching in an expression over another, so they can set up a style checker that warns about the use of such a pattern. These style checkers can also make use of VS Code’s suggested fix button in the hover view. This button can trigger another editor action, which replaces the code violating style rule with another piece of code with the preferred style.
- *Re-implementation of more existing interactions*: Given the possibility to reason about source locations, it is now possible to also re-implement the make case interaction (C-c C-c). In addition, it would also be possible to improve its functionality, by allowing it to create cases for lambda parameters for example.
- *Refactorings for dependently-typed programs*: We can also implement refactorings inspired from Barwell et al. [17]. They introduce refactorings such as adding an index to a data type definition, or relating two constructor parameters using a dependent type. This feature could be used to its full potential with bindings to VS Code’s refactorings menu. This menu allows the user to see a preview of the changes that a refactoring applies and to select multiple refactorings to see how they interact, and give the desired result.

TODO: Insert figure with VS Code refactoring menu

6. Conclusion

7. Future work

7.1. Verifying parts of the extension

7.2. Making the extension editor-agnostic

Looking at the implementations of existing Agda extensions, a lot of the infrastructure for handling user commands, and responses from the Agda process are quite similar (up to differences in programming languages). This “core” part of every extension can be extracted and generalised such that the editor interactions are abstract. With a working Visual Studio Code extension, we could try adding support for another IDE – for example Atom⁹, whose packages also need to be written in Javascript – to create an abstraction layer between the editor-specific implementations and the editor-agnostic Agda interfacing code. Additionally, if the “core” logic is designed in such a way that it can be compiled to both Haskell and Javascript, we could also add support for Neovim using the same library as Cornelis uses.

There are multiple way to design an editor-agnostic extension:

We can implement a WebAssembly target to Agda’s compilation stage. WebAssembly can be embedded in many different languages and runtimes, and therefore is an interesting target for the extension. For Visual Studio Code, we can use Node.JS’s API for interfacing with WebAssembly. The architecture is described as a diagram in Figure 22.

The downside to this is that the interfacing to the editor must be done outside of Agda. That is, the Foreign Function Interface that is currently implemented in Avea using compiler pragmas, should partly be moved to the host language.

Another option is to rework the extension to function as a language server. This means that it runs as a standalone program, which an editor-specific extension can spawn. The editor-specific extension only needs to implement the communication layer between the language server and the editor. The protocol used to communicate can be a custom protocol for Avea, or the custom extensions to LSP that Lean4 or Rocq have implemented could be implemented for Avea, as far as they make sense for Agda style interactions.

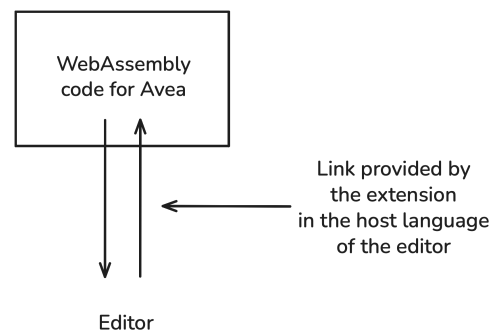


Figure 22: Hello world

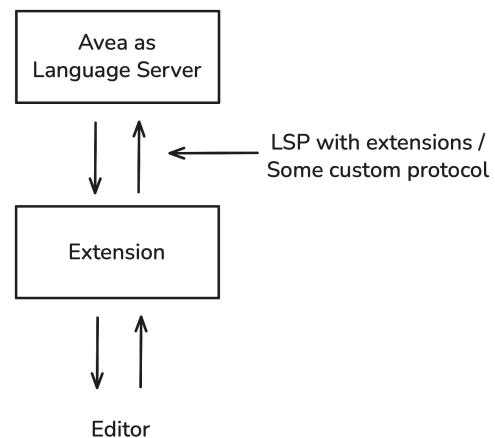


Figure 23: Hello world

7.3. Improving Agda’s Foreign Function Interface

During the development of the extension prototype thusfar, one major point of setback has been using the FFI. Currently, the FFI only provides one language pragma: `COMPILE JS`. This pragma takes a name of a function and the Javascript implementation for that function, and pastes it without semantic modification into the compiled output. The compiler has a flag `--js-verify` that checks for syntax

⁹<https://flight-manual.atom-editor.cc/hacking-atom/>

errors in the compiled output, but this gives very little correctness guarantees. This flag cannot, for instance, verify whether FFI functions have the correct number of parameters or the correct type.

We propose to improve the FFI by making it directed by Typescript functions. We can take a Typescript file and generate an Agda type for each exported function of the module we import using the FFI tool. For each function, we can request the parameter types and return type from the Typescript compiler, then create a curried function call and curried function type, and insert those terms into an Agda file. When going into this direction, we can adopt Typescript types incrementally:

1. We can select a subset of Typescript’s type system, for example akin to the types in System F limited to builtin types, that we want to convert to Agda. We would need to create an interface with the rather under-documented Typescript compiler API¹⁰ to analyse source files and request the types of functions within.
2. Once the basis for the tool works, we can try expanding the tool’s capabilities by generating ML-style modules for classes, and as such also try supporting types other than builtin types and type variables. This encoding for objects has been abundantly used in the extension already. Listing 32 shows roughly how a simple class in Typescript can be converted to a module in Agda. The constructor could be compiled to a function called `new`, every method to a function with the same name, and every type parameter to a declaration in a variable block.

<pre> 1 class List<T> { 2 constructor(length: number) {} 3 public get(index: number): T {} 4 } </pre> tsTypeScript	<pre> 1 module List where 2 private variable T : Set 3 postulate 4 t : Set → Set 5 new : Nat → t T 6 get : Nat → t T → T </pre> Agda
---	---

Listing 32: Proposed translation of class definitions in Typescript to modules in Agda.

Typescript’s type system has some features that Agda’s type system does not. For example, Typescript has unlabelled sum types: $T \mid U$. With Agda’s sum types $(T \uplus U)$, the left and right injection can still be discriminated, whereas in Typescript these types might not be discriminable. Another example is Typescript’s intersection types: $T \& U$, which creates a new object type that contain all fields that are common in T and U . Agda does not have row types, so finding the intersection of records is quite difficult. This leads us to believe that, even though research into this idea might be valuable, it is out of scope for this thesis.

¹⁰<https://github.com/Microsoft/TypeScript/wiki/Using-the-Compiler-API>

7.4. Port of Lean4's user widgets to Avea and Agda

As mentioned in Section 2.4.3, the `lean4-vscode` extension allows custom widgets to be embedded in the infoview. The architecture of Lean4's widgets can be described by the diagram in Figure 24 [14]. There are two points of special note:

1. Widgets can directly communicate via LSP to the Lean4 compiler. If the widgets want to evaluate metaprograms, then they can use the injected Lean4 RPC library to call metaprograms. The RPC library only allows function calls to metaprograms marked with the `@[server_rpc_method]` attribute. This attribute requires the function to be of type `A -> RequestM B` where `A` and `B` are JSON-serialisable.
2. The React code for the widgets is stored as a string in the Lean source file. When a widget should be loaded, this code is sent to the infoview, which embeds the React component into its own React app. The infoview manages all communication to and from the extension: if a message is native to the infoview, it handles it itself, otherwise it can dispatch the message to the widget.

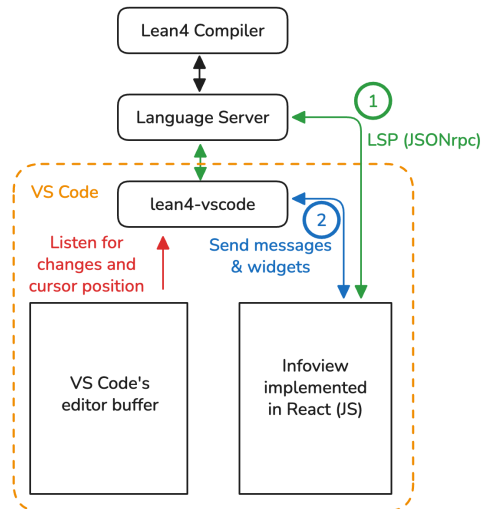


Figure 24: High-level architecture of Lean4's user widget system in `lean4-vscode`.

The same architecture can be taken inspiration from for Avea and Agda, a proposal for an architecture can be found in Figure 25. The main component of this architecture is `lepje`, an Elm-inspired web framework for Agda. Whereas widgets in Lean4 had to embed React code in the Lean source files, Avea would allow widgets to be written in Agda. Ideally, it would also allow the widgets to be co-located with the user's libraries or application code, similar to the edit-time functions. If the author of the widgets makes sure to write widgets that do not make use of FFI functions only defined for Haskell, then compiling an Agda project meant to work with the Haskell backend can be compiled to Javascript, and just pick out the widget code for running instead of, for example, the main function.

There are numerous use cases for more complex visualisations in the display buffer:

- A metaprogram can display an interactive dependency tree of a function to show information about the dependencies. This might include whether a dependency is a postulate without FFI definitions (useful for safety analysis), or ...

The infoview can be implemented in `lepje`, and its task will be to listen for new widgets and to handle all native messages from Avea (e.g. `DisplayInfo` and `RunningInfo` messages). The widgets themselves can also be implemented in `lepje`.

1. If a widget should be loaded, Avea compiles the widget via the Agda handle.
2. Agda puts the compiled JS code in a temporary folder, which Avea communicates to the infoview.
3. The infoview can create a new DOM element for the new widget and loads the widget's compiled code from the temporary folder.
4. The widget is initialised with RPC controls by the infoview to have two-way communication with the extension, and is allowed to use the newly created DOM node to display.
5. Communication always goes through Avea, since it already has an Agda session opened. So even if the widget

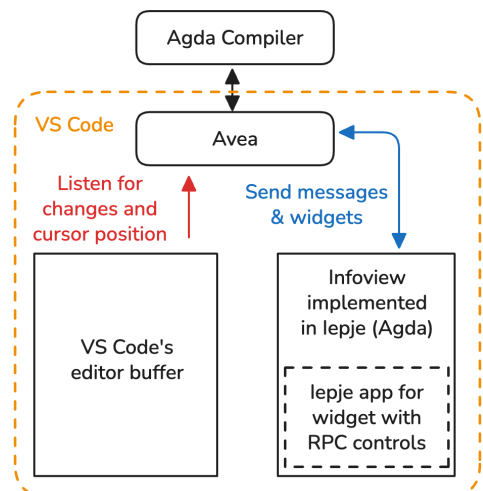


Figure 25: Architecture proposal for a version of Lean4's user widgets in Avea and Agda.

requests something from Agda, it should message Avea
that a message to the Agda process should be sent.

The advantage of this architecture is that, similar to the edit-time functions, all code can be written in just Agda. This means that writing widgets is more accessible for Agda library authors, and it communication between a widget and the extension can reuse the same types.

8. Bibliography

- [1] M. Dodds, ‘What Works (and Doesn’t) Selling Formal Methods’. Accessed: Mar. 13, 2026. [Online]. Available: <https://www.galois.com/articles/what-works-and-doesnt-selling-formal-methods>
- [2] S. Juhošová, A. Zaidman, and J. Cockx, ‘Pinpointing the learning obstacles of an interactive theorem prover’, in *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, 2025, pp. 159–170.
- [3] S. Juhošová, A. Zaidman, and J. Cockx, ‘The Way of Types: A Report on Developer Experience with Type-Driven Development’, in *International Conference on Program Comprehension (ICPC)*. ACM. <https://doi.org/10.2026>.
- [4] R. Backx, ‘A Grounded Theory Study of "How Interactive Theorem Prover Programmers Write Code"’, Master’s thesis, 2026.
- [5] Stack Overflow, ‘Stack Overflow Developer Survey 2025’. Accessed: Feb. 26, 2025. [Online]. Available: <https://survey.stackoverflow.co/2025/>
- [6] Microsoft, ‘language-server-protocol’. Accessed: Oct. 05, 2022. [Online]. Available: <https://microsoft.github.io/language-server-protocol/>
- [7] G. Lomurno and idris2-lsp contributors, ‘idris2: Language Server for Idris2’. Accessed: Dec. 28, 2025. [Online]. Available: <https://github.com/idris-community/idris2-lsp>
- [8] L. de Moura and S. Ullrich, ‘The Lean 4 Theorem Prover and Programming Language’, in *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*, Berlin, Heidelberg: Springer-Verlag, 2021, pp. 625–635. doi: 10.1007/978-3-030-79876-5_37.
- [9] E. Tassi, R. Tetley, and vsrocq contributors, ‘vsrocq: Visual Studio Code extension for Coq’. Accessed: Nov. 27, 2025. [Online]. Available: <https://github.com/rocq-prover/vsrocq>
- [10] E. J. Gallego Arias and rocq-lsp contributors, ‘Rocq-lsp: Visual Studio Code Extension and Language Server Protocol for Rocq’. Accessed: Nov. 30, 2025. [Online]. Available: <https://github.com/ejgallego/rocq-lsp>
- [11] J. K. Rask, F. P. Madsen, N. Battle, H. D. Macedo, and P. G. Larsen, ‘The Specification Language Server Protocol: A Proposal for Standardised LSP Extensions’, in *Proceedings of the 6th Workshop on Formal Integrated Development Environment, F-IDE@NFM 2021, Held online, 24-25th May 2021*, J. Proença and A. Paskevich, Eds, in EPTCS, vol. 338. 2021, pp. 3–18. doi: 10.4204/EPTCS.338.3.
- [12] D. Christiansen and E. Brady, ‘Elaborator reflection: extending Idris in Idris’, in *Proceedings of the 21st acm sigplan international conference on functional programming*, 2016, pp. 284–297.
- [13] J. Korkut and D. T. Christiansen, ‘Extensible type-directed editing’, in *Proceedings of the 3rd ACM SIGPLAN International Workshop on Type-Driven Development*, in TyDe 2018. St. Louis, MO, USA: Association for Computing Machinery, 2018, pp. 38–50. doi: 10.1145/3240719.3241791.
- [14] W. Nawrocki, E. W. Ayers, and G. Ebner, ‘An Extensible User Interface for Lean 4’, in *14th International Conference on Interactive Theorem Proving (ITP 2023)*, A. Naumowicz and R. Thiemann, Eds, in Leibniz International Proceedings in Informatics (LIPIcs), vol. 268. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, p. 24:1–24:20. doi: 10.4230/LIPIcs.ITP.2023.24.
- [15] P. D. van der Walt, ‘Reflection in agda’, Master’s thesis, 2012.

- [16] W. Kokke and W. Swierstra, ‘Auto in agda: programming proof search using reflection’, in *International Conference on Mathematics of Program Construction*, 2015, pp. 276–301.
- [17] A. D. Barwell, C. Brown, M. S. Chang, C. Theocharis, and S. Thompson, ‘Structural Refactorings for Exploring Dependently Typed Programming’, in *Trends in Functional Programming: 25th International Symposium, TFP 2024, South Orange, NJ, USA, January 10–12, 2024, Revised Selected Papers*, South Orange, NJ, USA: Springer-Verlag, 2024, pp. 1–21. doi: 10.1007/978-3-031-74558-4_1.